

1. 프로젝트 개요

한 줄 요약 · RoomieSync는 함께 사는 룸메이트들의 **가사 당번 자동 로테이션**과 **공동 지출 자동 정산**을 한 곳에서 해결하는 협업형 iOS 앱이다.

1.1 문제 정의

자취·기숙사·세어하우스 거주자가 늘면서, 룸메이트 간 두 가지 분쟁이 반복적으로 발생한다.

- **가사 분담 불균형** — 누가 언제 무엇을 했는지 기록되지 않아 "내가 더 많이 한다"는 인식이 쌓이고 갈등으로 이어진다.
- **공동 지출 정산 누락** — 휴지·세제·공과금 등 소액을 카톡으로 주고받다 누락·중복·계산 오류가 빈번하고 송금도 늦어진다.

기존 대안(카톡 메모, 가계부 앱, 종이 당번표)은 **공유성**과 **자동화** 중 하나가 빠져 실사용으로 이어지지 않는다.

1.2 타겟 사용자 & 기대 효용

사용자 페르소나	페인 포인트	RoomieSync의 가치
김지훈 (24, 대학생) 친구 2명과 자취 3년차	"내가 항상 쓰레기 버리는 것 같은데"라는 의심, 산 휴지 값 까먹고 못 받는 일	자동 당번표로 객관화, 영수증 한 번 입력으로 정산 자동화
박서연 (22, 세어하우스) 처음 만난 4명과 동거	아직 친하지 않아 가사·돈 얘기 꺼내기 어려움	앱이 알아서 알림, 직접 말 꺼낼 필요 없음

평가지표 매핑 — 효용성: 실사용자(자취생) 검증 가능한 페인을 2개 정조준. 두 기능이 결합되어야만 풀리는 문제라는 점에서 단일 기능 앱 대비 차별화.

1.3 경쟁 앱 비교

앱	가사 관리	지출 정산	룸메이트 공유	RoomieSync의 차별점
Splitwise	✗	✓	✓	단일 앱에서 가사 + 지출 + 통계 를 통합 관리하며, iOS 17의 위젯·Live Activity로 "오늘 한 일"을 매 순간 확인 가능
Tody (가사 전용)	✓	✗	부분	
똑똑가계부	✗	✓	✗	
카카오톡 메모	△	△	✓	

1.4 성공 지표 (KPI)

지표	목표	측정 방법
그룹당 주 평균 가사 완료율	≥ 80%	ChoreCompletion 로그 / 예정 횟수
공정 지수 (멤버 간 가사 분담 균형도)	≥ 85%	$\min(\text{멤버별 완료 횟수}) / \max(\text{멤버별 완료 횟수}) \times 100$
지출 → 정산 완료까지 평균 시간	≤ 7일	Expense.createdAt → Settlement.settledAt
7일 리텐션	≥ 60%	가입일 + 7일 시점 앱 실행 여부
위젯 활성 사용자 비율	≥ 40%	WidgetCenter 호출 빈도 추적

2. 핵심 기능

우선순위	기능	상세 동작
P0 MVP	① 룸메이트 그룹 생성·초대	그룹 생성 → 초대 코드(6자리) 발급 → 멤버 입장. 최대 6인까지 한 그룹.
P0	② 가사 당번 자동 로테이션	가사 항목(청소·설거지·쓰레기)별 주기 설정 → 멤버 순서대로 자동 배정 → 당일 푸시 → 완료 체크 → 다음 차례로 이동
P0	③ 공동 지출 입력·정산	항목·금액·결제자·참여자 입력 → 멤버별 잔액(받을/줄 돈) 자동 계산 → 정산 완료 처리
P1	④ 활동 히스토리 & 통계	최근 30일 누가 어떤 가사를 몇 회 했는지, 누적 지출 그래프
P2	⑤ 토스/카카오페이 송금 딥링크	정산 금액 그대로 송금 앱 열기 (URL Scheme, 외부 앱 위임)

※ P0는 시연 영상 필수 포함, P1은 시간 허용 시 구현, P2는 확장 항목.

3. 시스템 아키텍처

3.1 전체 구조 (MVVM + CloudKit)

► Presentation Layer (SwiftUI Views)

OnboardingView · GroupHomeView · ChoreListView · ExpenseListView · StatsView · SettingsView

▶ ViewModel Layer (@Observable / Combine)

GroupViewModel · ChoreViewModel · ExpenseViewModel · SettlementCalculator

▶ Domain Layer (Pure Swift)

Models (Group, Member, Chore, Expense) · Use Cases (RotateChore, CalculateBalance, SettleDebts)

▶ Data Layer (Repository Pattern)

ChoreRepository · ExpenseRepository · GroupRepository Local: SwiftData (오프라인 캐시) ↔ Remote: CloudKit (공유 동기화)

▶ Platform Services

UserNotifications (당번 알림) · CloudKit Sharing (그룹 공유 DB) · WidgetKit (오늘의 당번 위젯) · URL Scheme (송금 앱)

평가지표 매핑 — 완결성: MVVM 분리로 레이어별 테스트 가능, Repository 패턴으로 데이터 소스 교체 가능. 실무 표준 구조.

3.2 데이터 모델 (SwiftData)

Group

id · name · inviteCode · createdAt · members[]

Member

id · name · avatarColor · joinedAt

Chore

id · title · icon · cycleType · currentAssigneeID ·
nextDueDate · history[]

Expense

id · title · amount · paidBy · participants[] · date ·
isSettled

ChoreCompletion

id · choreID · memberID · completedAt

Settlement

fromMember · toMember · amount · settledAt

3.3 핵심 알고리즘 — 채무 단순화 (Debt Simplification)

각 멤버의 순잔액을 계산한 뒤 그리디 매칭으로 채권자·채무자를 연결하여 송금 횟수를 최소화한다.

1. 각 멤버 $net_balance = (받을 돈) - (줄 돈)$ | 2. 양수 큐(채권자) · 음수 큐(채무자) 분리·정렬
3. 두 큐의 head끼리 $\min(|채권|, |채무|)$ 만큼 송금 매칭 | 4. 0이 된 쪽 제거, 반복
- 최악의 경우 N-1번 송금으로 정산 종료 (N=멤버 수)

4. 알림 · 위젯 · Live Activity — "오늘 한 일" 확인 시스템

앱을 열지 않아도 "오늘 내 차례 뭐였지? 했나?"를 즉시 확인할 수 있어야 한다는 핵심 사용자 동기에 맞춰, iOS 17의 플랫폼 기능 3종을 다층적으로 활용한다.

4.1 홈/잠금화면 위젯 (WidgetKit)

위젯 크기	표시 내용	사용 시나리오
Small (홈)	오늘 내 차례 가사 1개 + 완료 상태 (예: "🗑️ 쓰레기 · 아직")	홈 화면에서 한눈에 "내 할 일 남았나?" 확인
Medium (홈)	룸메이트 전원의 오늘 진행 상황 체크리스트 (이름 + ✓/🕒)	"누가 했고 누가 안 했나" 한눈에 파악, 무언의 압박 효과
Lock Screen (iOS 16+)	오늘 미완료 가사 수 (예: "남은 가사 2")	잠금화면 들춰볼 때마다 자연스럽게 인지
앱 아이콘 배지	오늘 미완료 가사 수 (빨간 숫자)	홈 화면에서 앱 아이콘만 봐도 인지

4.2 푸시 알림 전략 (UserNotifications)

시점	알림 내용	대상 / 정책
당일 오전 9:00	"오늘 쓰레기 당번이에요 🗑️"	당번 본인에게만, 시간대는 사용자 설정 가능
당일 오후 9:00	"아직 쓰레기 안 하셨네요. 한 번 확인해보세요"	미완료자에게만 (조용한 리마인드 톤)
룸메이트 완료 시	"박서연이 설거지를 완료했어요 ✓"	그룹원 전체, 설정에서 OFF 가능 (피로도 방지)
지출 입력 시	"김지훈이 휴지 9,900원을 등록했어요"	참여자 전원, 무음 알림 옵션
월말 정산	"이번 달 정산할 시간이에요. 받을 돈 12,500원"	매월 마지막 날 오전 10시

4.3 Live Activity & Dynamic Island (iOS 16.1+)

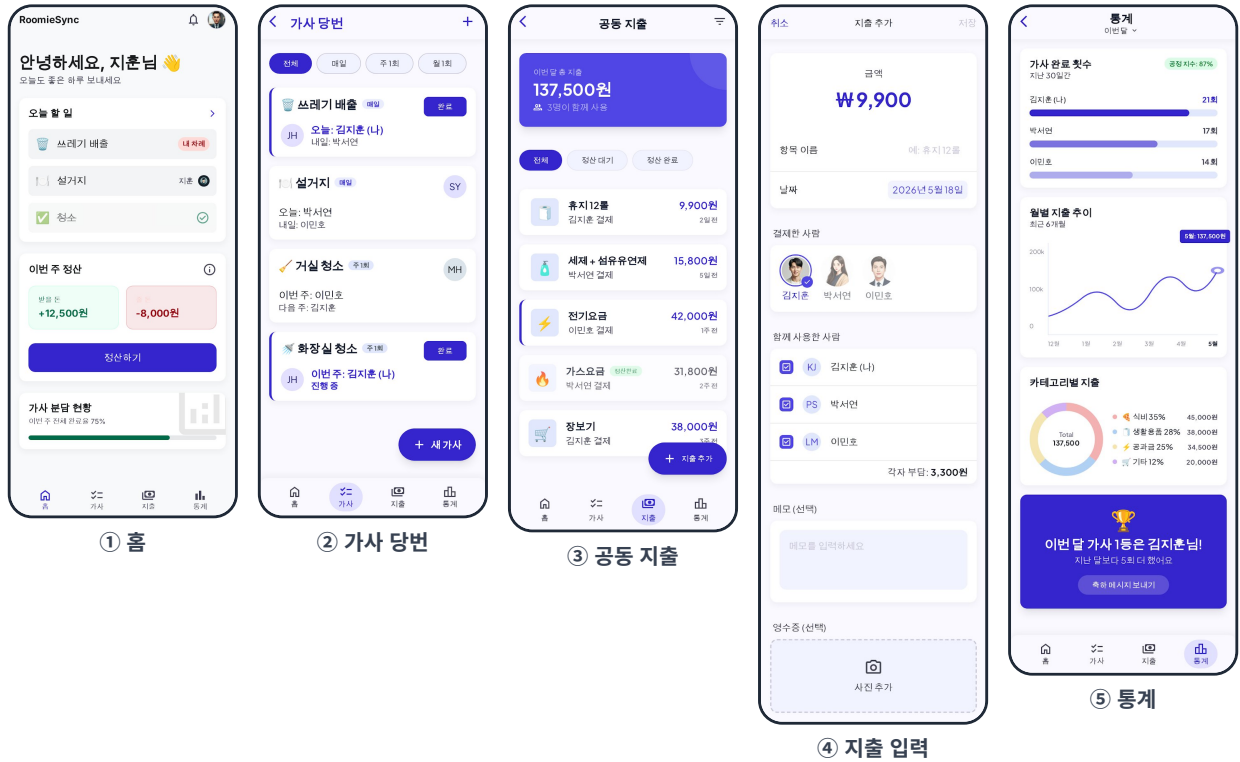
가사 진행 중 활동

가사 "시작" 버튼 누르면 Live Activity 시작 → 잠금화면 · Dynamic Island에 "🗑️ 쓰레기 배출 중..." 표시 → "완료" 버튼 한 번에 종료. 굳이 앱을 다시 열지 않아도 됨.

정산 카운트다운 활동

월말 3일 전부터 Live Activity로 "정산까지 D-3" 표시. 멤버 전원에게 정산 임박을 시각적으로 자연스럽게 인지시킴.

5. UI/UX 와이어프레임



5.1 디자인 시스템

Primary Color	#4F46E5 (Indigo 600) — 행동 유도 버튼 · 활성 탭
Semantic Colors	완료 #10B981 · 오늘 차례 #EF4444 · 정산 대기 #F59E0B
Typography	SF Pro 시스템 — Title 28pt Bold · Body 17pt · Caption 13pt
컴포넌트	iOS Native 우선 (List, NavigationStack, TabView, Form, Chart). Human Interface Guidelines 준수
접근성	Dynamic Type 지원 · VoiceOver 레이블 · 색맹 대응 (색 + 아이콘 이중 표현)

평가지표 매핑 — 시각디자인 · 학습용이성 · 라벨링: 단일 Primary 컬러로 일관성, 한국어 직관 라벨(우리집/가사/지출), 색+아이콘+텍스트의 3중 표현으로 상태 즉시 파악 가능.

6. 사용자 시나리오 & UX 디테일

6.1 핵심 사용자 여정

#	상황	사용자 행동 · 시스템 반응
1	최초 실행 (호스트)	이름·아바타 선택 → 그룹 생성 → 초대코드 6자리 발급 → 클립보드 자동 복사
2	룸메이트 합류	앱 설치 → 초대코드 입력 → 그룹 자동 참여 → CloudKit으로 데이터 즉시 공유
3	가사 세팅	"청소·설거지·쓰레기" 템플릿에서 선택 또는 직접 추가 → 주기 설정 → 멤버 가입 순으로 자동 순환
4	일상 사용	아침 9시 푸시 "오늘 쓰레기 당번이에요" → 완료 체크 → 자동으로 다음 멤버로 이동
5	지출 발생	마트에서 휴지 구매 → "지출 추가" → 금액·결제자·참여자 입력 → 잔액 자동 갱신
6	월말 정산	"정산하기" → 채무 단순화로 최소 송금안 제시 → 토스 딥링크로 즉시 송금

6.2 평가지표별 UX 설계 의도

평가 항목	설계 의도 & 구현 방식
직관성	홈 진입 시 "오늘 할 일" + "내 잔액"만 노출, 1초 안에 핵심 정보 전달. 액션은 + 버튼 한 곳으로 통일
라벨링	"홈/가사/지출/통계" 명확한 한글 동사·명사. 외래어·약어 배제. "정산 완료/대기" 상태 라벨 명시
피드백	가사 완료 시 햅틱(.success) + 애니메이션 + 토스트. 지출 저장 시 잔액 카운트업 애니메이션
오류 정정	금액 0원이면 저장 비활성화 · 참여자 0명 시 경고 · 가사 완료 후 5초 "취소" 토스트 · 그룹 삭제는 확인 다이얼로그 + 텍스트 입력 요구
학습용이성	첫 실행 시 3장 온보딩(문제→솔루션→사용법). 빈 상태(Empty State) 화면마다 "다음에 뭐 할지" 안내 문구

6.3 엣지 케이스 처리

- 오프라인 — SwiftData 로컬 저장 후 네트워크 복귀 시 CloudKit 자동 동기화 (충돌 시 last-write-wins)
- 멤버 탈퇴 — 미정산 잔액 있으면 정산 완료 강제, 미정산 시 탈퇴 차단
- 당번 회피 — "오늘 못해요" 버튼 → 다음 멤버에게 스왑 요청 → 수락 시 순서 교체 기록
- 지출 수정·삭제 — 정산 완료된 지출은 수정 불가 (감사 추적성 확보)

6.4 테스트 시나리오 (XCTest)

테스트 대상	시나리오 & 검증 항목
채무 단순화 알고리즘	3~4인 케이스 각 5개씩. 모든 멤버 net_balance 합이 정확히 0, 송금 횟수 ≤ N-1 검증
가사 로테이션 로직	멤버 추가/제거 시 순서 무결성, "당번 회피" 후 다음 차례 정합성
CloudKit 동기화 충돌	두 기기에서 같은 가사 동시 완료 → last-write-wins로 한쪽만 기록
오프라인 → 온라인 복귀	오프라인 중 입력한 지출 3건이 복귀 후 모든 멤버 기기에 동기화되는지

7. 기술 스택

분류	선택 기술	선택 이유
언어 / UI	Swift 5.9 · SwiftUI · @Observable	과목 주제 부합, iOS 17 최신 매크로 활용
로컬 저장	SwiftData	CoreData 대비 보일러플레이트 적고 CloudKit 연동 간편
동기화 / 백엔드	CloudKit (Private + Shared DB)	서버 구축 불필요, 무료, 룸메이트 공유 DB 기본 제공
알림	UserNotifications (Local)	로컬 알림으로 충분, APNs 인증서 불필요
차트	Swift Charts (iOS 16+)	네이티브 차트 API, 외부 라이브러리 없음
송금 연동	Toss / KakaoPay URL Scheme	실제 송금은 외부 앱 위임 (PG 인증 불필요)
의존성 관리	Swift Package Manager	외부 의존성 최소화로 빌드 안정성 확보

8. 개발 일정 (4주)

작업	1주차 5/19~5/25	2주차 5/26~6/1	3주차 6/2~6/8	4주차 6/9~6/14
설계 · 데이터 모델링 · 시안	●●●●●			
P0 - 그룹 · 가사 로테이션	●●	●●●●●		
P0 - 지출 · 채무 단순화		●●	●●●●	
CloudKit 동기화 · 푸시 알림			●●●●●	
P1 - 통계 · 위젯 · Live Activity			●●	●●●
XCTest 단위 테스트 작성		●	●●	●●
버그 픽스 · 영상 촬영 · README				●●●●●

9. 리스크 & 대응

리스크	영향	대응 전략
CloudKit Sharing 설정 복잡성	높음	1주차에 PoC 우선. 실패 시 Firebase Firestore로 대체
실제 룸메이트 테스트 어려움	중간	시뮬레이터 2대 + 실기기 1대로 다중 사용자 시연, 친구 1명 TestFlight 배포
채무 단순화 알고리즘 버그	중간	3~5인 케이스 XCTest 단위 테스트, 잔액 합이 0인지 검증
3분 영상에 기능 다 못 담기	중간	10.1 시나리오 사전 작성: 문제 → 그룹 → 가사 → 지출 → 차별점 → 마무리 6단계로 시간 배분 고정
일정 지연	낮음	P0 완료를 3주차로 잡고 P1·P2는 잘라낼 수 있게 모듈 분리

10. 시연 영상 & 산출물

10.1 3분 영상 시나리오

시간	장면	내레이션 · 화면
0:00~0:25	문제 제기	"매번 누가 쓰레기 버릴 차례인지 까먹어서 싸운 적 있으신가요?" — 카톡 캡처 대비
0:25~0:50	그룹 생성 → 초대	호스트 + 룸메이트 화면 분할로 실시간 동기화 시연
0:50~1:30	가사 로테이션	당번 알림 → 완료 체크 → 다음 멤버 자동 이동까지 풀 플로우
1:30~2:20	지출 → 자동 정산	4인 시나리오, 채무 단순화로 송금 횟수가 줄어드는 효과 강조
2:20~2:50	통계 · 차별점	"단순 가계부 X, 단순 당번표 X — 두 기능 결합으로 진짜 분쟁이 사라진다"
2:50~3:00	마무리	GitHub QR + 다운로드 안내

10.2 GitHub Repository 구조

```
roomiesync/
├── README.md ← 프로젝트 소개 메인 (아래 10.3 참조)
├── docs/
│   ├── architecture.png ← MVVM + CloudKit 다이어그램
│   ├── erd.png ← SwiftData 엔티티 관계도
│   ├── screens/ ← 화면별 스크린샷 (5종)
│   └── demo.gif ← 핵심 플로우 GIF (3MB 이하)
├── RoomieSync/ ← Xcode 프로젝트
│   ├── App/ ← RoomieSyncApp.swift (@main)
│   ├── Features/ ← Group · Chore · Expense · Stats 각 모듈
│   ├── Core/ ← Domain, Repository, Services
│   ├── Widget/ ← WidgetKit Extension
│   ├── LiveActivity/ ← ActivityKit Extension
│   └── Resources/ ← Assets, Localizable.strings
├── RoomieSyncTests/ ← XCTest 단위 테스트
├── .github/workflows/ci.yml ← GitHub Actions (빌드 검증)
└── LICENSE ← MIT
```

10.3 README.md 구성 (정보성 강화)

섹션	내용
① 헤더 (최상단)	앱 로고 · 한 줄 소개 · 데모 GIF (3초 핵심 플로우) · 시연 영상 임베드 + YouTube 링크 · GitHub QR
② 문제 정의	3문장 이내. "왜 만들었나"를 카톡 캡처 예시와 함께
③ 주요 기능	각 기능 1문장 + 스크린샷 1장씩 (5개 화면)
④ 기술 스택	배지(shields.io) 형태로 시각화: Swift · SwiftUI · SwiftData · CloudKit 등
⑤ 아키텍처	MVVM 다이어그램 이미지 + 폴더 구조 트리
⑥ 빌드 방법	Xcode 버전, iOS 타겟, CloudKit 컨테이너 설정, 빌드 명령 4단계
⑦ 핵심 알고리즘	채무 단순화 의사코드 + 예시 입출력
⑧ 테스트	XCTest 실행 방법, 커버리지 캡처
⑨ 향후 계획	Android 버전, AI 가사 추천, B2B 세어하우스 등
⑩ 라이선스 & 연락	MIT, 이메일

평가지표 매핑 — 정보성: README 최상단 "데모 GIF + 1줄 소개"로 5초 안에 이해 가능, 시연 영상은 임베드 + YouTube 이중 제공, QR로 모바일 접근성 확보. 10개 섹션으로 평가자가 원하는 정보를 즉시 찾을 수 있는 구조.

11. 참고문헌 & 레퍼런스

분류	출처
Apple 공식 문서	Apple Human Interface Guidelines (developer.apple.com/design/human-interface-guidelines) · SwiftData Documentation · CloudKit Sharing Guide · WidgetKit · ActivityKit · UserNotifications
아키텍처 패턴	Martin Fowler, "Patterns of Enterprise Application Architecture" (Repository Pattern) · Robert C. Martin, "Clean Architecture" (Layer 분리 원칙)
핵심 알고리즘	Splitwise Engineering Blog, "Simplifying Debts" — 채무 단순화 그리디 매칭 접근법 · Cormen et al., "Introduction to Algorithms" 3rd ed., Chapter 16 (Greedy Algorithms)
경쟁 앱 분석	Splitwise (App Store, 2026.05 기준) · Tody · 똑똑가계부 · Mobbin 디자인 라이브러리
디자인 시안	Stitch by Google (stitch.withgoogle.com) — UI 와이어프레임 생성 도구 · SF Symbols 5 · iOS Design Resources
레퍼런스 코드	Apple Sample Code "Synchronizing a Local Store to the Cloud" · Hacking with Swift — SwiftData & CloudKit Tutorial Series